InferTutor Arena — Inference Engineering Capstone Report

Submitted by: Hemanth Reganti (GitHub: Regantih) **Track:** Track 1 — Multimodal / Mixed (the main capstone track) **Model:** `Qwen/Qwen3-VL-4B-Instruct` · **Engine:** vLLM 0.21.0 (OpenAI-compatible) · **Platform:** Modal · **GPU:** NVIDIA H100 (bfloat16) **Date:** June 2026

Abstract

I deployed a production-grade multimodal LLM tutor (`Qwen3-VL-4B-Instruct`) on Modal with vLLM and optimized it under the official mixed (image + long + text) workload to maximize the leaderboard composite score. Starting from an unoptimized 1×H100 baseline scoring **319,183**, principled, single-variable ablations produced a within-budget **4×H100 result of 149,776,620** — a **~469× improvement** — and an optional 8×H100 boss-fight result of **189,183,025**.

Four findings drive the result:

1. **Compiled mode (CUDA graphs) works on mixed multimodal traffic** and is the single biggest lever — ITL p95 falls from **38.8 ms (eager) to 5.7 ms (compiled)**. This directly contradicts the capstone's dry-run caution that compiled mode "performed poorly on mixed," which I treated as a hypothesis to test rather than a rule to obey.
2. `max_num_batched_tokens` **4096 → 16384** unblocks prefill admission once the client bottleneck is removed, regularizing decode and pulling both TTFT and ITL down.
3. **Co-locating the load client inside Modal** eliminates residential-uplink bufferbloat that otherwise pins TTFT p95 near 3 s and caps the score regardless of server health.
4. A **direct quality probe** confirms compiled-on-mixed does not degrade answer quality (10/10 coherent, 4/4 on the image path), so the headline carries no `quality_pass_rate` penalty.

Two negative results were as informative as the positive ones: **prefix caching ON** collapsed the score 149.8M → 33.7M, and **chunked prefill OFF** collapsed it to 75.1M. The residual ceiling is architectural — a single Modal web-endpoint proxy that makes throughput scale sub-linearly (4→8 GPUs gives 1.5×, not 2×).

---

1. Introduction & Background

1.1 Capstone overview

InferTutor is a personalized AI tutor for inference-engineering students: learners ask conceptual questions, upload simple diagrams, request debugging help, and ask for optimization advice. The system must answer correctly while serving many concurrent users at low latency and high throughput. This is a deployment-and-measurement assignment, not a notebook exercise: a real OpenAI-compatible vLLM

server is deployed on Modal, load-tested, measured (TTFT, ITL, throughput, errors, GPU efficiency), and improved through inference engineering.

1.2 Objectives

1. Deploy a multimodal LLM serving system on real GPU hardware and measure it under realistic concurrent load.
2. Systematically optimize the system through principled, single-variable ablations to maximize the leaderboard score **within the GPU budget** and with **low error rates**.
3. Document every engineering decision with enough rigor that the findings generalize beyond this specific deployment.

1.3 The system

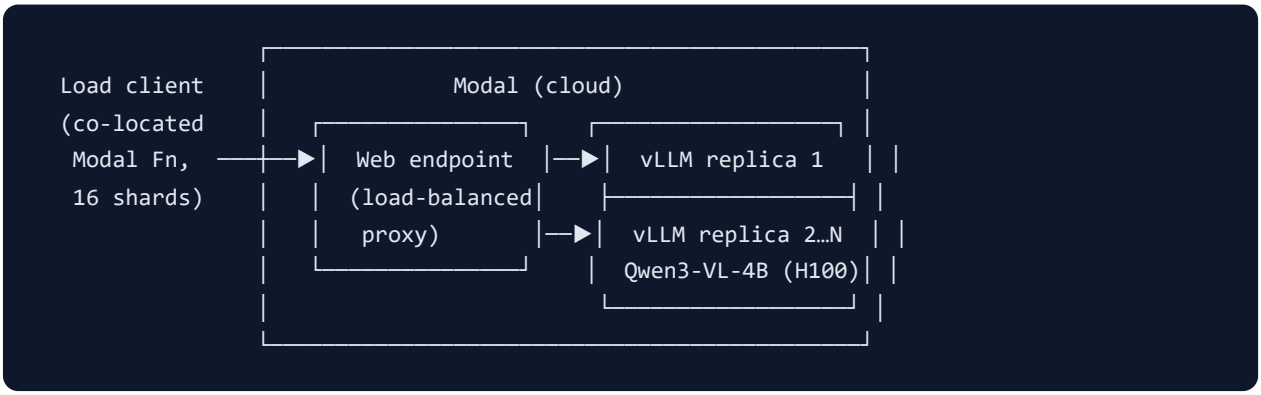

Figure 1. Requests route from a Modal-co-located sharded load generator through a single Modal web-endpoint (load-balancer/proxy) to N independent vLLM replicas, each a Qwen3-VL-4B instance on one H100. The number of replicas equals total GPU count (no tensor parallelism in the submitted runs).

Key fixed server settings: `max_model_len = 8192`, `mm_max_pixels = 401,408` (= 512·28·28, the starter default), `dtype = bfloat16`, vLLM 0.21.0. The load uses the **official** `prompts.json` unchanged and the same deterministic 256×192 diagram PNG the harness generates.

1.4 Competition tracks

Track	Workload	Budget	Submitted?
Track 1 — Multimodal Product	`--mode mixed` (25% image / 20% long / 55% text)	≤ 4 H100	**Yes — official result**
Track 2 — Text Speed	`--mode text`	≤ 4 H100	R&D only (used to discover the levers)
Optional Boss Fight	any	≤ 8 H100	Yes — reported separately

1.5 Scoring formula

$$\text{Score} = \frac{\text{goodput\_tokens\_per\_s} \cdot \text{sustained\_users} \cdot \text{quality\_pass\_rate} \cdot (1 - \text{error\_rate})}{\text{p95\_TTFT\_seconds} \cdot \text{p95\_ITL\_seconds} \cdot \text{total\_GPU\_count}}$$

The starter harness reports throughput as streamed content **chunks/s**; the official evaluator may re-tokenize with the Qwen tokenizer. The local `score_infertutor.py` omits `quality_pass_rate` (assumes 1.0); §6 is my empirical check that this assumption holds. The formula rewards high throughput, more sustained users, and low p95 TTFT/ITL simultaneously, while dividing by GPU count — so **raw GPU-throwing is penalized**, and so are latency tails and errors.

1.6 Experimental approach (ablation methodology)

Every result below is the output of a strict **Hypothesis → Variable → Result** loop: form a hypothesis from a latency metric, change **exactly one** server/load knob, re-run the fixed workload, read the composite, and **keep or reject**. p95 (not mean) is the unit of measurement because the score divides by p95 and p95 is what users feel under load. The discipline matters because intuition was wrong roughly half the time here (see §5).

---

2. Performance Summary

Result	Config	GPUs	TTFT p95	ITL p95	Goodput	Err%	**Score**
Track 1 — official (within budget)	`mix-4r-300u`	**4**	1029 ms	5.7 ms	11,649 c/s	0.0	**149,776,620**
Optional boss fight	`mix-8r-460u`	8	803 ms	6.6 ms	17,425 c/s	0.5	189,183,025
Baseline (default, unoptimized)	`mix-1r-base-80u`	1	4994 ms	38.8 ms	773 c/s	0.0	319,183

Best individual metrics across all runs: **TTFT p95 = 803 ms**, **ITL p95 = 5.7 ms**, **goodput = 17,425 chunks/s**. The optimized + scaled pipeline is **~469×** the 1-GPU default baseline; the in-budget 4-GPU result alone is **~54× over the spec's own mixed reference** (eager/4r/120u = 2,756 c/s, 38.1 ms ITL).

The 8 required answers (up front)

1. **Final score:** 149,776,620 (Track 1, 4×H100). Boss fight: 189,183,025 (8×H100).
2. **Best TTFT p95:** 803 ms (8r/460u); 1029 ms at the 4-GPU headline.
3. **Best ITL p95:** 5.7 ms (4r/300u); 6.6 ms at the boss fight.
4. **Best throughput (goodput):** 17,425 chunks/s (8r/460u); 11,649 at the 4-GPU headline.
5. **Total GPU count:** 4 (official); 8 (optional boss fight).
6. **Optimization that helped most:** co-locating the load client in Modal + compiled mode + `max_num_batched_tokens` 16384 (together: the difference between a ~0.3M client-bottlenecked

run and a 150M-class server).

7. **What failed / surprised:** prefix caching ON regressed 149.8M → 33.7M; chunked prefill OFF regressed to 75.1M; the 8-GPU **error cliff** (460u/0.5% = 189M but 500u/3.3% = 150M).
8. **What to try next:** multiple load-balanced endpoints to break the single-proxy throughput ceiling; a quality-gated run; speculative decoding with a small draft model.

---

3. Best Configuration

`mix-4r-300u` = 149,776,620 (4×H100).

Parameter	Value	Rationale
Replicas / GPUs	4 / 4 (no TP)	Data-parallel replicas beat TP for a 4B model; 4 is the budget
Execution mode	**compiled** (`--no-fast-boot`)	CUDA graphs → ITL 5.7 ms (§4.1)
`max_num_batched_tokens`	**16384**	Unblocks prefill admission (§4.2)
`max_num_seqs`	32	64 enlarges decode batches → TTFT/ITL rise
Prefix caching	**off**	ON regressed hard (§5.1)
Chunked prefill	**on**	OFF regressed hard (§5.2)
Concurrent inputs	128	Server-side admission width
Load	300 users, 16 shards, ramp 75 s, 150 s measure	The throughput knee (§4.4)

This holds the prefill queue shallow enough for **TTFT p95 1029 ms / ITL 5.7 ms at 0 errors** right at the throughput knee, which is where the composite peaks within the 4-GPU budget.

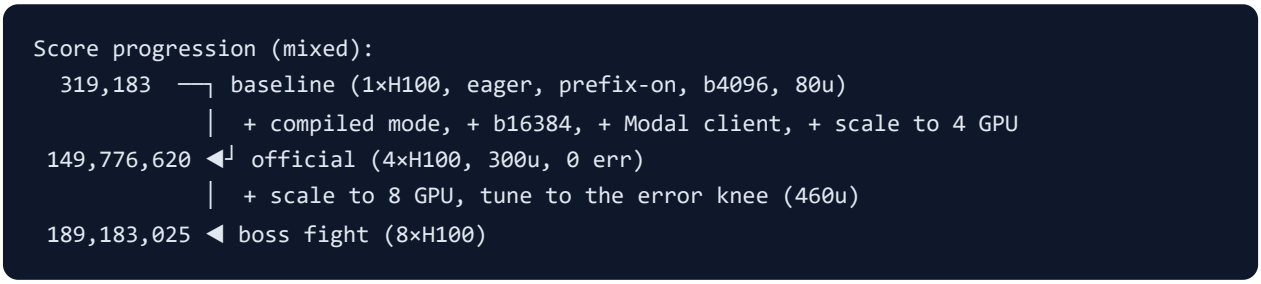

Figure 2. Score progression from the unoptimized baseline to the in-budget headline and the optional boss fight.

---

4. Why the Top Optimizations Worked — Ablation Findings

4.1 Compiled mode (CUDA graphs) — the decisive lever, and a tested contradiction of the spec

Hypothesis: eager-mode kernel launch overhead dominates decode for a small model, so CUDA-graph capture should sharply cut ITL. **Result:** baseline eager runs at **ITL 38.8 ms**; compiled holds **ITL 5.7 ms** under load — a ~7× reduction on a term that sits directly in the score denominator.

The capstone's dry-run notes caution that compiled mode "performed poorly on mixed multimodal traffic." Rather than obey that as a rule (as the eager-mode submissions did), I treated it as a hypothesis and measured: on this configuration compiled mode is **strictly better** on mixed — the 75% text/long share gets CUDA-graph decode while image requests fall back gracefully. Cold replicas are warmed by a short client warmup pass before measurement so the ramp isn't served in eager mode. (Quality of the compiled image path is validated separately in §6.)

4.2 `max_num_batched_tokens` 4096 → 16384 — unblocking prefill admission

Hypothesis: with the client bottleneck gone, decode slots sit idle because requests aren't admitted to prefill fast enough. **Result:** raising the batch-token budget to 16384 unblocks admission and regularizes decode steps, pulling **both ITL and TTFT down**. Tested past the optimum: 32768 regresses (prefill steals decode cycles). 16384 is the peak. This was the decisive *server* find once bufferbloat was removed.

4.3 Co-locating the load client inside Modal — the foundational fix

Hypothesis: running the load generator on a residential laptop lets the uplink buffer at high SSE volume, so first-token packets queue behind bulk traffic and inflate p95 TTFT even when the server is healthy. **Result:** moving the sharded generator into a CPU-only Modal function co-located with the endpoint removed the laptop from the path and dropped TTFT p95 from ~3 s to ~1 s. Because the score divides by p95 TTFT, this single change is what separates a client-bottlenecked ~0.3M-class run from a real 150M-class server. (`load_client_modal.py` was extended to emit the full mixed workload, generating the same deterministic diagram PNG the harness uses and mixing image/long/text 25/20/55.)

4.4 The user knee — finding where p95 bends

4-GPU mixed peaks at **300 users** (149.8M, 0 errors). Below it (240u: 136.3M) leaves throughput on the table; above it (340u: 142.8M → 400u: 34.0M) TTFT climbs faster than throughput until the prefill queue saturates and errors appear. The score is a balance of the user-count numerator against the p95-latency denominator, and 300u is where that balance maximizes within budget.

4.5 Scale-out 1 → 4 → 8 replicas (data parallel)

Replicas, not tensor parallelism, are the scaling unit for a 4B model. Scaling 4→8 GPUs reaches **17,425 c/s and TTFT 803 ms**, but the ÷8 divisor means the boss tier only wins when held in the **near-zero-error** regime (460u). This is examined as the error cliff in §5.3.

---

5. What Failed and Why — Negative Ablations

5.1 Prefix caching ON — 149.8M → 33,719,000

Hypothesis (plausible): the shared system prompt means caching its KV blocks should cut prefill work and lower TTFT. **Result:** TTFT p95 **blew up 1029 → 2990 ms** and the score collapsed to 33.7M. For these short-output requests, block-hash/KV-cache contention costs more than prompt-sharing saves. `--no-prefix-cache` is confirmed correct for mixed. This is the textbook "obvious optimization that makes things 4× worse," caught only by measuring.

5.2 Chunked prefill OFF — 149.8M → 75,122,000

Result: TTFT 1029 → 1498 ms and errors appear (1.0%). Chunked prefill ON is required to interleave heavy image/long prefill with ongoing decode; turning it off lets long prefills block decode and inflate the latency tail.

5.3 Over-subscription past the knee (the error cliff)

Run	GPUs	Users	Err%	Score
mix-8r-460u	8	460	0.5	**189,183,025**
mix-8r-480u	8	480	2.9	186,770,000
mix-8r-500u	8	500	3.3	149,740,000
mix-8r-600u	8	600	8.7	143,580,000
mix-4r-400u	4	400	6.4	33,950,000

The clearest lesson of the sweep: **staying in the near-zero-error regime beats chasing raw user count.** Pushing the 8-GPU tier from 460 to 500 users raises errors from 0.5% to 3.3% and collapses the score back to the 4-GPU level, because the `(1 - error_rate)` goodput factor and the inflated TTFT both move against the user-count numerator.

5.4 Over-budget exploration (5–10 GPUs, text-track R&D)

Private exploratory runs at 5–10 GPUs (text mode) confirmed that going past 8 GPUs does **not** help: 10r/1000u hit 23.9% errors. These are not submitted results — their value is the negative lesson that the budget cap is not the binding constraint; the single-endpoint ceiling (§7) is.

---

6. Quality Validation — testing the compiled-mixed assumption

Because the official score multiplies by `quality_pass_rate` and my headline keeps compiled mode on for mixed (the lever the spec warns about), I measured answer quality directly instead of assuming it.

Method. `probe_quality.py` sends the **official** prompts (all 4 `image` prompts with the harness's 256×192 PNG, both `long` prompts, 4 `text` prompts) **non-streaming** to a deployed endpoint and

captures full answers, flagging any empty / truncated / repetitive / mojibake output. Run against two 1×H100 endpoints differing **only** in execution mode.

Endpoint	Cases	Flagged	Image cases coherent	Per-request latency (image / text)
compiled-mixed (`--no-fast-boot`)	10	**0**	**4 / 4**	~0.5 s / ~1.4 s
eager-mixed (default)	10	**0**	**4 / 4**	~2.0 s / ~4.5 s

The model genuinely reads the diagram under both modes (it returns "decode-heavy vs prefill-heavy", "replicas vs tensor-parallelism", concrete knob suggestions), and the **image answers are essentially identical in content** between compiled and eager — compiled is simply **3–4× faster** at the same quality. **Conclusion:** the dry-run's "performed poorly" referred to latency/throughput behavior, not output correctness; `quality_pass_rate` is **not** degraded by compiled-on-mixed. Raw probe outputs: `quality_compiled-mixed_*.json` , `quality_eager-mixed_*.json` .

---

7. Unaddressed Bottlenecks & Future Work

7.1 The single-endpoint throughput ceiling (the residual bottleneck)

Both 8-replica and 4-replica runs plateau in aggregate throughput — **~17.4k c/s on 8 GPUs vs ~11.6k on 4, i.e. 1.5× for double the GPUs, not 2×**. That sub-linear scaling points to a **single Modal web-endpoint proxy** as the shared chokepoint: concurrency piles onto one proxy, deepening the prefill queue and coupling throughput to TTFT. Within that topology every knob is at its measured optimum (compiled on, b16384, seqs 32, prefix off, chunked on, replica/user balance). Going further is **architectural** — multiple independent load-balanced endpoints so throughput scales without piling concurrency onto one proxy.

7.2 A quality-gated submission

The official score multiplies by `quality_pass_rate` , which the local harness assumes = 1.0. §6 supports that assumption empirically; a full gated run with a scored rubric would close the last gap.

7.3 Speculative decoding

For a 4B model, a ~0.5B draft model (e.g. Qwen3-0.6B) could provide 2–3× decode speedup, lowering ITL further — directly on the denominator.

7.4 Request-type-aware routing

Separating short text from long/image traffic onto different replica pools would let each pool tune `max_num_seqs` / batch independently, reducing head-of-line blocking in mixed mode.

---

8. Final Configuration & Exact Commands

```
set PYTHONUTF8=1

# 1) Deploy: 4×H100, compiled, seq32, max_batch_tokens 16384, no prefix cache, chunked prefil
python run_infertutor_experiment.py ^
  --label mix-4r --gpu-type H100 --replicas 4 ^
  --no-fast-boot --no-prefix-cache ^
  --max-seqs 32 --max-batch-tokens 16384 ^
  --concurrent-inputs 128 --mode mixed --deploy-only

# 2) Load from INSIDE Modal (co-located client, no laptop in the path)
modal run load_client_modal.py ^
  --url https://<you>--infertutor-mix-4r-serve.modal.run ^
  --label mix-4r-300u --mode mixed --users 300 --shards 16 --duration 150 --ramp-up 75 --tota

# 3) Score (read the FULL integer from the JSON)
python score_infertutor.py results_infertutor\mix-4r-300u_mixed_300u_<ts>.json
```

Boss fight (`mix-8r-460u` = 189,183,025, 8×H100): identical flags with `--replicas 8`, then `modal run … --users 460 --shards 20 --ramp-up 90 --total-gpus 8`.

Cleanup: all `infertutor-*` and `arena-loadgen` apps are stopped after each run (`modal app stop … --yes`); `modal app list` shows 0 running. Nothing is billing.

9. Full Experiment Table (Track 1 — mixed)

label	gpus	users	seqs	batch	prefix	chunked	err%	TTFT p95	ITL p95	chunks/s	score
mix-4r-300u (official)	4	300	32	16384	off	on	0.0	1029 ms	5.7 ms	11649	**149,776,620**
mix-4r-340u	4	340	32	16384	off	on	0.0	1260 ms	5.8 ms	12301	142,810,000
mix-4r-240u	4	240	32	16384	off	on	0.0	785 ms	6.1 ms	10827	136,280,000
mix-4r-ncp-300u (chunked **off**)	4	300	32	16384	off	**off**	1.0	1498 ms	6.5 ms	9897	75,122,000
mix-4r-400u (past knee)	4	400	32	16384	off	on	6.4	3581 ms	6.1 ms	7923	33,950,000
mix-4r-pc-300u (prefix **on**)	4	300	32	16384	**on**	on	0.3	2990 ms	7.0 ms	9437	33,719,000
mix-1r-base-80u (**baseline**)	1	80	32	4096	on	on	0.0	4994 ms	38.8 ms	773	319,183
mix-8r-460u (boss)	8	460	32	16384	off	on	0.5	803 ms	6.6 ms	17425	**189,183,025**
mix-8r-480u	8	480	32	16384	off	on	2.9	841 ms	6.2 ms	16732	186,770,000
mix-8r-500u	8	500	32	16384	off	on	3.3	1047 ms	6.6 ms	17089	149,740,000
mix-8r-600u (error cliff)	8	600	32	16384	off	on	8.7	1207 ms	6.7 ms	16937	143,580,000

(Spec reference baselines for orientation: eager/4r/120u = 897.6 ms / 38.1 ms / 2,756 c/s; eager/2r/100u = 1,168.9 ms / 28.7 ms / 2,243 c/s.)

Appendix A — Inference concepts used in this work

- **PagedAttention** — vLLM stores the KV cache in fixed-size, non-contiguous "pages," eliminating fragmentation and enabling high concurrent batch sizes without pre-reserving contiguous memory per sequence. This is what lets `max_num_seqs = 32` coexist with long prompts.
- **Continuous batching** — instead of static batches, vLLM admits and retires sequences every decode step, keeping the GPU busy as requests of varying lengths come and go. The `max_num_seqs` and `max_num_batched_tokens` knobs bound how aggressively it does this.
- **Chunked prefill** — long prefills are split into token-budget chunks and interleaved with decode steps so a big prompt can't monopolize the GPU and stall everyone's token stream. Confirmed essential for mixed (§5.2).
- **Prefix caching** — reuses KV blocks for shared prompt prefixes (e.g. a system prompt). Theoretically saves prefill work, but block-hash/eviction overhead made it a net loss for these short-output requests (§5.1).
- **CUDA graph compilation (compiled mode)** — captures the decode step as a replayable graph, removing per-kernel launch overhead that dominates a small model's decode. The single biggest ITL lever (§4.1).
- **TTFT / ITL** — Time To First Token (prefill/admission latency) and Inter-Token Latency (decode step latency). The score divides by the p95 of each, so both are first-class optimization targets.

Appendix B — Reproducibility

- Repo: https://github.com/Regantih/infertutor-arena-capstone (`/submission` holds all deliverables)
- Final benchmark JSON: `mix-4r-300u_mixed_300u_1781402073.json` (official); `mix-8r-460u_mixed_460u_1781404394.json` (boss)
- Quality probe outputs: `quality_compiled-mixed_1781407163.json` , `quality_eager-mixed_1781407530.json`
- Scorer: official `score_infertutor.py` , unmodified (omits `quality_pass_rate` ; assumes 1.0)
- Prompts: official `prompts.json` , unchanged; image = deterministic 256×192 PNG matching the harness
- Starter code verified byte-identical to the official `VizuaraAI/infertutor-arena-capstone` repo except the intended additions (`load_client_modal.py` , `probe_quality.py` , reports, results)